

SWE40006 - Software Deployment and Evolution

Deployment Portfolio Task 2

Student Name: Nguyen Huyen Minh Nhat
Student ID: 105550225
Unit: SWE40006 - Software Deployment and Evolution
Semester: Fall 2026
Due Date: 05 October 2026, 23:59
Submission Date: 05 October 2026

Table of Contents

1. Task Level Attempted	2
2. Introduction	2
3. Initial EC2 and WordPress Deployment	2
4. External Database, Load Balancer and Backup/Restore	5
5. Application Load Balancer	10
6. Launch Template and Auto Scaling	11
7. Windows SSH Access and CLI Verification	14
8. Errors and Resolutions	15
a) Error 01 - SSH key path not found	15
b) Error 02 - WordPress database connection failure	15
c) Error 03 - ALB endpoint initially unreachable	16
d) Error 04 - Incorrect CLI command	16
9. Public Verification Endpoint and Deployment Resources	17
10. Source Code and Deployment Commands	17
11. Conclusion	17

1. Task Level Attempted

I am attempting Task 2 at Level 2.4 (High Distinction). The work includes the requirements of Levels 2.1, 2.2 and 2.3, with EC2, RDS, S3, Elastic Load Balancing, Launch Template, Auto Scaling Group and Windows SSH access demonstrated.

2. Introduction

The WordPress application was deployed on Amazon Web Services using an EC2 instance and then extended to use an external Amazon RDS database, an Application Load Balancer and Auto Scaling. The backup and restore requirement was also completed, and the final deployment was verified through the ALB endpoint and command-line access from Windows PowerShell.

The main public verification endpoint is: <http://t2-swe40006-alb-2041814639.us-east-1.elb.amazonaws.com>

3. Initial EC2 and WordPress Deployment

An EC2 key pair named T2_SWE40006_key and security group T2_SWE40006_SG were created. The security group allowed SSH access from the working client IP and HTTP access for the WordPress service. The WordPress application was installed on an Amazon Linux EC2 instance and verified through Apache.

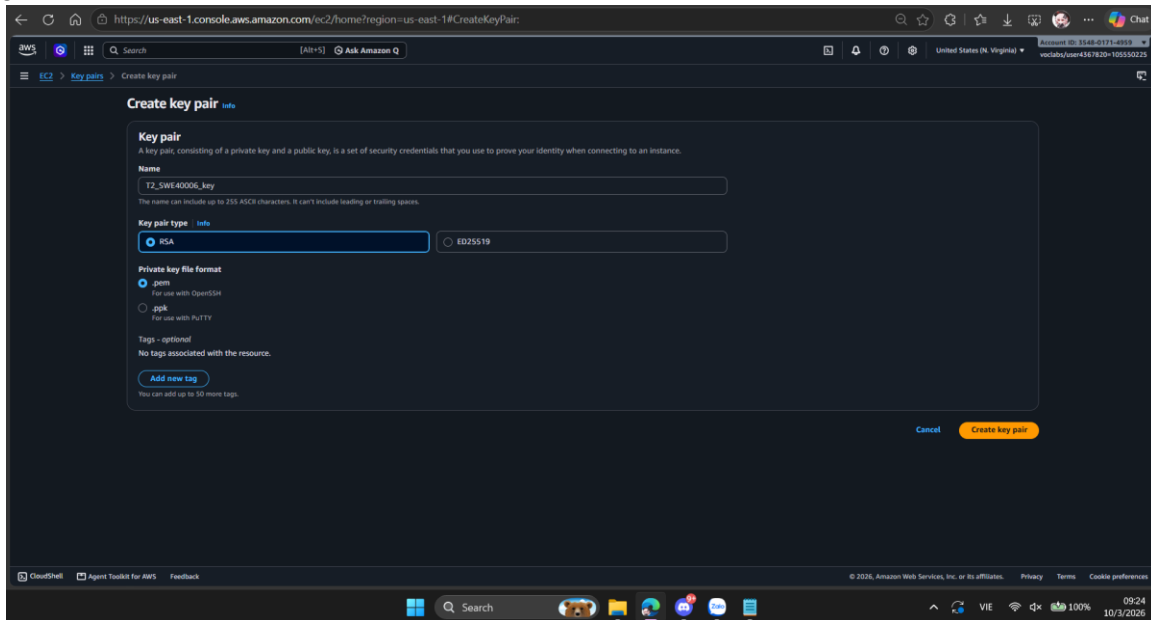


Fig. 1. EC2 SSH key pair T2_SWE40006_key created for instance access.

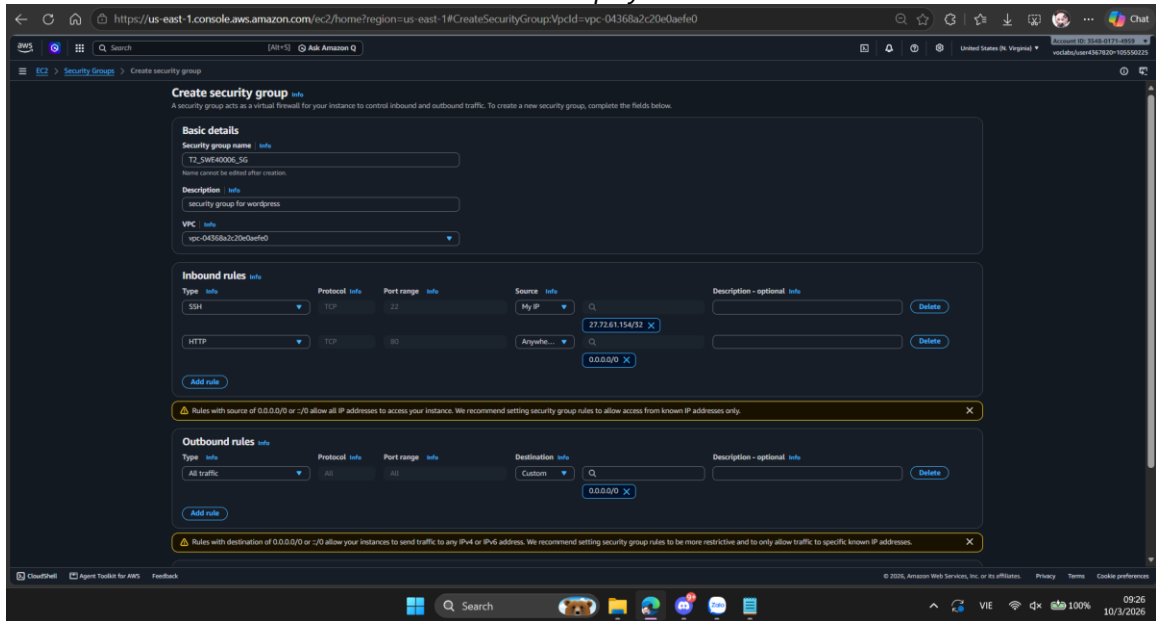


Fig. 2. EC2 security group configured for SSH and HTTP access.

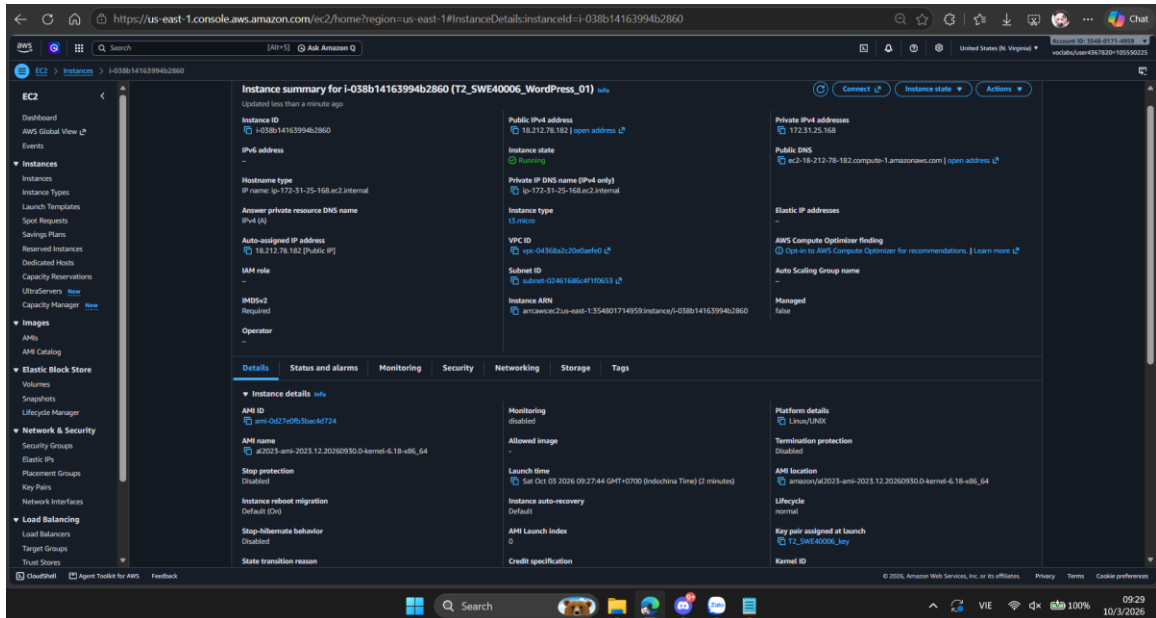


Fig. 3. Initial WordPress EC2 instance configuration.

```

Windows PowerShell
|-----|
| 27588 /usr/sbin/httpd -DFOREGROUND |
| 27589 /usr/sbin/httpd -DFOREGROUND |
| 27611 /usr/sbin/httpd -DFOREGROUND |
| 27613 /usr/sbin/httpd -DFOREGROUND |

Oct 03 03:20:16 ip-172-31-25-168.ec2.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Oct 03 03:20:16 ip-172-31-25-168.ec2.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Oct 03 03:20:16 ip-172-31-25-168.ec2.internal httpd[27587]: Server configured, listening on: port 80
[ec2-user@ip-172-31-25-168 ~]$ curl -I http://localhost
HTTP/1.1 403 Forbidden
Date: Sat, 03 Oct 2026 03:21:06 GMT
Server: Apache/2.4.68 (Amazon Linux)
Last-Modified: Fri, 07 Nov 2025 08:23:08 GMT
ETag: "bf-642fce432f308"
Accept-Ranges: bytes
Content-Length: 191
Content-Type: text/html; charset=UTF-8

[ec2-user@ip-172-31-25-168 ~]$ ls -la /var/www/html
total 0
drwxr-xr-x. 2 root root 6 Jun 9 06:02 .
drwxr-xr-x. 4 root root 32 Oct 3 03:18 ..
[ec2-user@ip-172-31-25-168 ~]$ echo '<html><body><h1>SWE40006 Apache Server</h1><p>Web server is running successfully.</p></body></html>' | sudo tee /var/www/html/index.html
<html><body><h1>SWE40006 Apache Server</h1><p>Web server is running successfully.</p></body></html>
[ec2-user@ip-172-31-25-168 ~]$ curl -I http://localhost
HTTP/1.1 200 OK
Date: Sat, 03 Oct 2026 03:22:54 GMT
Server: Apache/2.4.68 (Amazon Linux)
Last-Modified: Sat, 03 Oct 2026 03:22:47 GMT
ETag: "64-65ce729b644fc"
Accept-Ranges: bytes
Content-Length: 169
Content-Type: text/html; charset=UTF-8

[ec2-user@ip-172-31-25-168 ~]$ curl.exe -I http://18.212.78.182
-bash: curl.exe: command not found
[ec2-user@ip-172-31-25-168 ~]$ exit
logout
Connection to 18.212.78.182 closed.
PS C:\WINDOWS\system32> curl.exe -I http://18.212.78.182
HTTP/1.1 200 OK
Date: Sat, 03 Oct 2026 03:24:40 GMT
Server: Apache/2.4.68 (Amazon Linux)
Last-Modified: Sat, 03 Oct 2026 03:22:47 GMT
ETag: "64-65ce729b644fc"
Accept-Ranges: bytes
Content-Length: 169
Content-Type: text/html; charset=UTF-8

PS C:\WINDOWS\system32>

```

Fig. 4. WordPress successfully served through the EC2 public endpoint.

The initial deployment provided the base EC2 WordPress environment required for the higher task levels.

4. External Database, Load Balancer and Backup/Restore

The EC2 deployment was extended by moving the WordPress database to Amazon RDS. The RDS endpoint was tested from the EC2 instance before the migration. The existing WordPress database was exported with mysqldump, imported into RDS and then configured as the external database in wp-config.php. The migrated database was verified by checking the WordPress tables, posts and user records.

```
[ec2-user@ip-172-31-25-168 ~]$ nc -zv t2-swe40006-rds.cxmbzgtkcaz.us-east-1.rds.amazonaws.com 3306
-bash: nc: command not found
[ec2-user@ip-172-31-25-168 ~]$ sudo grep -E "DB_NAME|DB_USER|DB_HOST" /var/www/html/wp-config.php
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'wpuser' );
define( 'DB_HOST', 'localhost' );
[ec2-user@ip-172-31-25-168 ~]$ mariadb -h t2-swe40006-rds.cxmbzgtkcaz.us-east-1.rds.amazonaws.com -P 3306 -u wpuser -p wordpress
Enter password:
ERROR 1045 (28000): Access denied for user 'wpuser'@'172.31.25.168' (using password: YES)
[ec2-user@ip-172-31-25-168 ~]$ nc -zv t2-swe40006-rds.cxmbzgtkcaz.us-east-1.rds.amazonaws.com 3306
-bash: nc: command not found
[ec2-user@ip-172-31-25-168 ~]$ sudo dnf install -y nmap-ncat
Last metadata expiration check: 1 day, 4:57:16 ago on Sat Oct 3 03:17:29 2026.
Dependencies resolved.
=====
Package                               Architecture      Version           Repository        Size
=====
Installing:
nmap-ncat                             x86_64            3:7.93-4.amzn2023  amazonlinux      225 k
=====
Transaction Summary
-----
Install 1 Package

Total download size: 225 k
Installed size: 478 k
Downloading Packages:
nmap-ncat-7.93-4.amzn2023.x86_64.rpm                                     8.7 MB/s | 225 kB  00:00
-----
Total
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing      :
  Installing     : nmap-ncat-3:7.93-4.amzn2023.x86_64
  Running scriptlet: nmap-ncat-3:7.93-4.amzn2023.x86_64
  Verifying      : nmap-ncat-3:7.93-4.amzn2023.x86_64
Installed:
nmap-ncat-3:7.93-4.amzn2023.x86_64

Complete!
[ec2-user@ip-172-31-25-168 ~]$ nc -zv t2-swe40006-rds.cxmbzgtkcaz.us-east-1.rds.amazonaws.com 3306
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: Connected to 172.31.29.128:3306.
Ncat: 0 bytes sent, 0 bytes received in 0.02 seconds.
[ec2-user@ip-172-31-25-168 ~]$
```

Fig. 5. Connectivity from EC2 to the RDS MySQL endpoint on port 3306.

```
-bash: !Secure: event not found
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'wpuser' );
define( 'DB_PASSWORD', 'WpSWE40006@2026' );
define( 'DB_HOST', 't2-swe40006-rds.cxmbzgtkcaz.us-east-1.rds.amazonaws.com' );
[ec2-user@ip-172-31-25-168 ~]$ sudo sed -i 's#define( '\''DB_PASSWORD'\'' ,*#define( '\''DB_PASSWORD'\'' , '\''WpSWE40006_2026!Secure'\'' );# ' /var/www/html/wp-config.php
grep -E "DB_NAME|DB_USER|DB_PASSWORD|DB_HOST" /var/www/html/wp-config.php
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'wpuser' );
define( 'DB_PASSWORD', 'WpSWE40006_2026!Secure' );
define( 'DB_HOST', 't2-swe40006-rds.cxmbzgtkcaz.us-east-1.rds.amazonaws.com' );
[ec2-user@ip-172-31-25-168 ~]$ sudo systemctl restart httpd
curl -I http://localhost
HTTP/1.1 200 OK
Date: Sun, 04 Oct 2026 08:30:30 GMT
Server: Apache/2.4.68 (Amazon Linux)
X-Powered-By: PHP/8.5.10
Link: <http://34.234.97.25/index.php/wp-json/>; rel="https://api.w.org/"
Content-Type: text/html; charset=UTF-8
[ec2-user@ip-172-31-25-168 ~]$ mariadb -h t2-swe40006-rds.cxmbzgtkcaz.us-east-1.rds.amazonaws.com -P 3306 -u wpuser -p wordpress
Enter password:
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 188
Server version: 10.11.18-MariaDB-log managed by https://aws.amazon.com/rds/
Copyright (c) 2008, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [wordpress]> SELECT COUNT(*) AS post_count FROM wp_posts;
+-----+
| post_count |
+-----+
|          5 |
+-----+
1 row in set (0.001 sec)

MariaDB [wordpress]> SELECT COUNT(*) AS user_count FROM wp_users;
+-----+
| user_count |
+-----+
|          1 |
+-----+
1 row in set (0.001 sec)

MariaDB [wordpress]>
```

Fig. 6. RDS WordPress database verification showing the migrated tables and records.

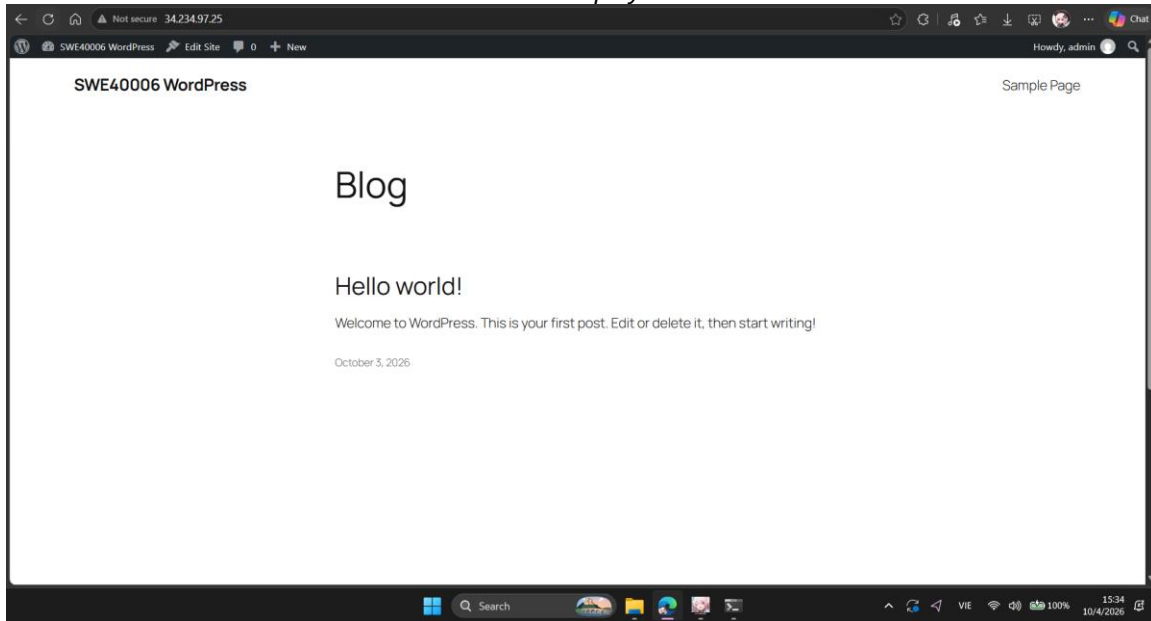


Fig. 7. WordPress successfully operating after migration to the external RDS database.

For the backup and restore requirement, an S3 bucket was created in the same AWS region. An EC2 AMI backup was then created and monitored until it became available. A new EC2 instance was launched from the available backup AMI.

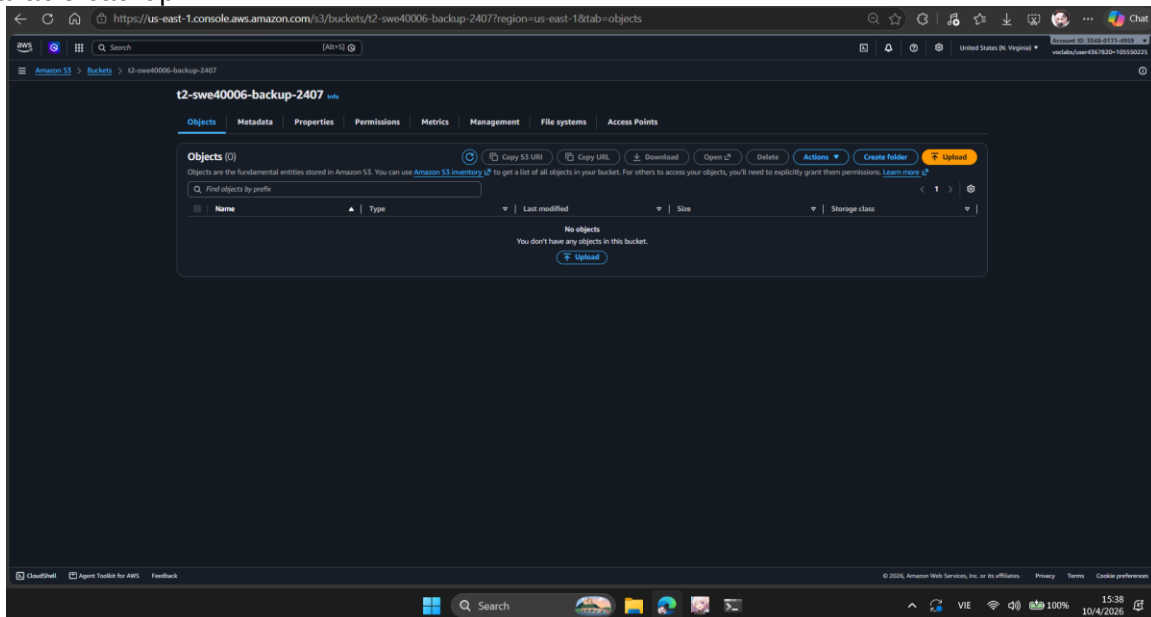


Fig. 8. S3 backup bucket in the AWS region.

SWE40006 - Software Deployment and Evolution

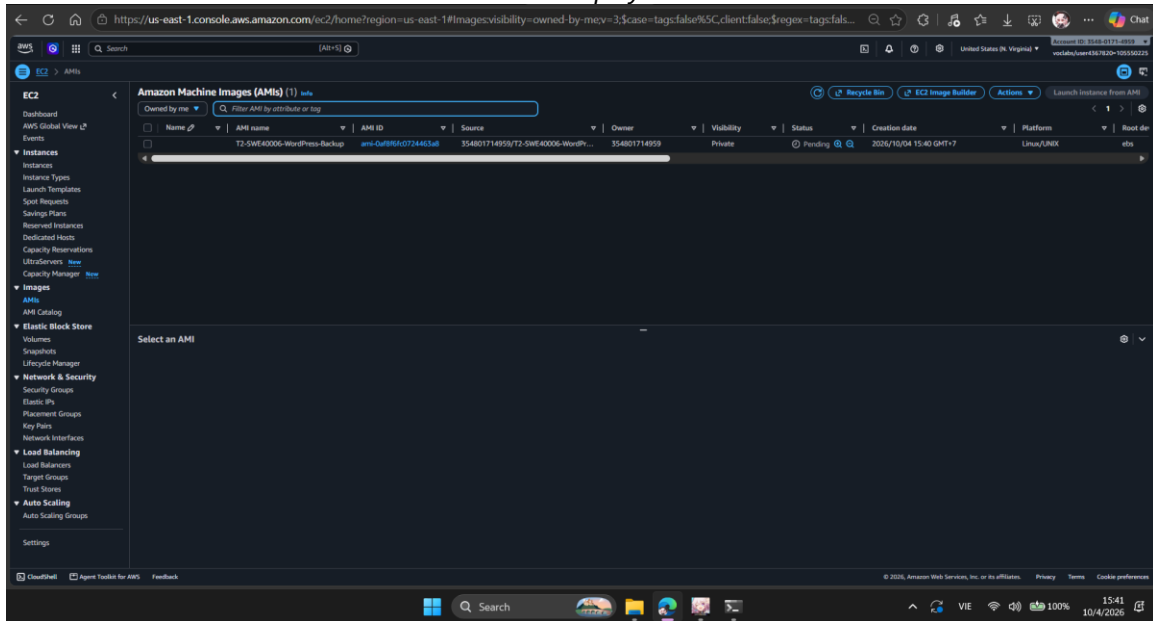


Fig. 9. EC2 backup AMI while the backup operation was in progress.

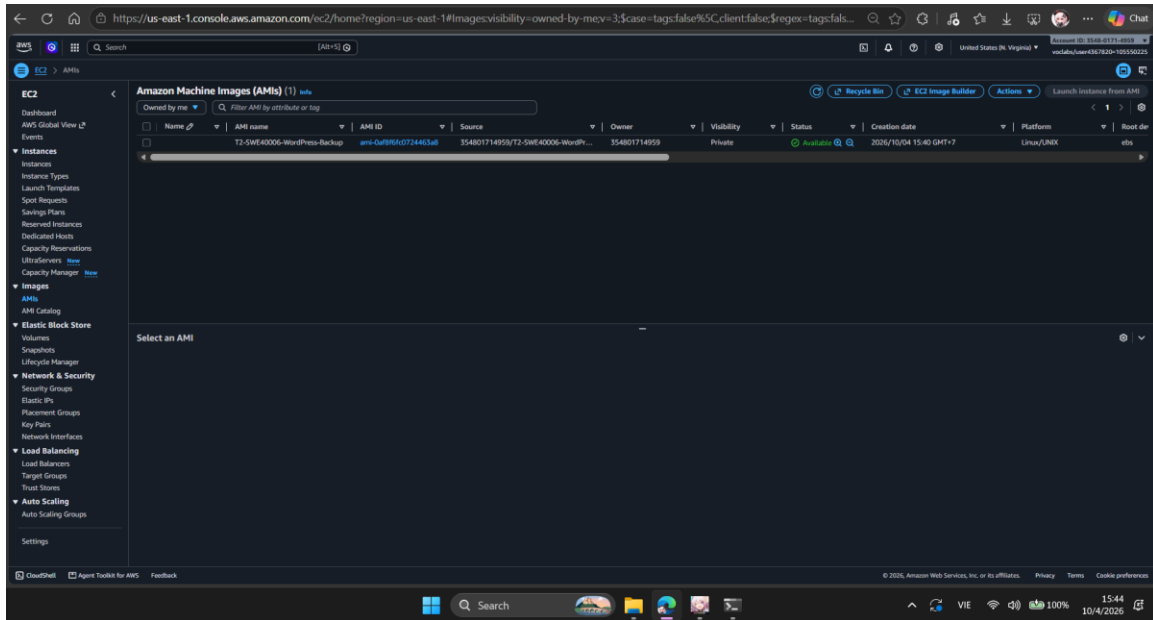


Fig. 10. EC2 backup AMI became available for recovery.

SWE40006 - Software Deployment and Evolution

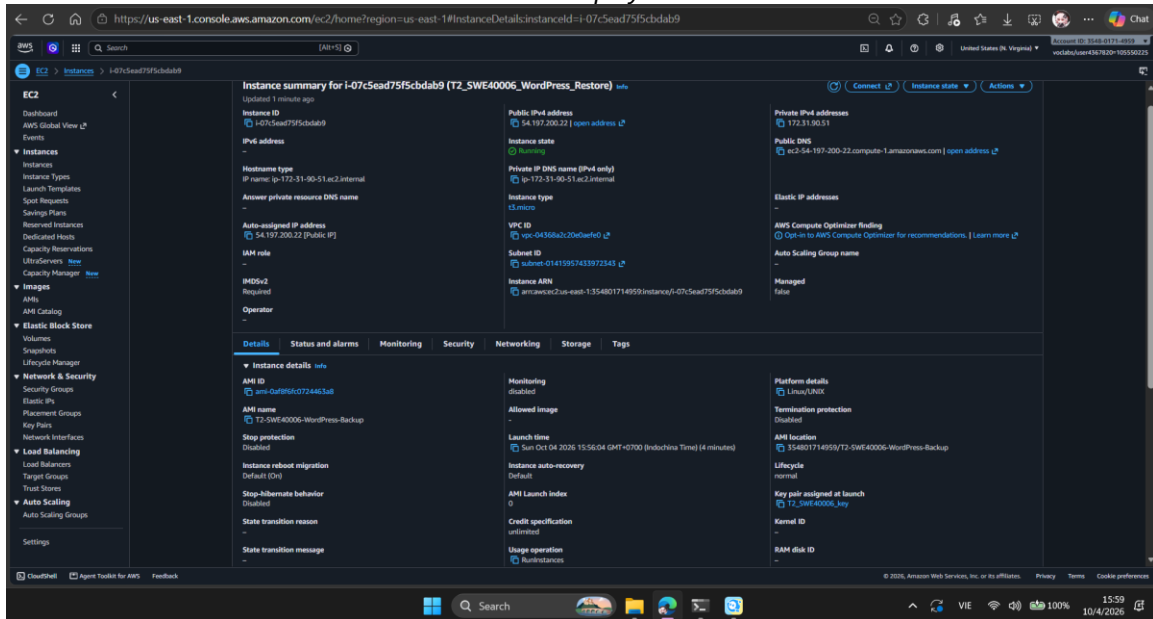


Fig. 11. New WordPress EC2 instance restored from the backup AMI.

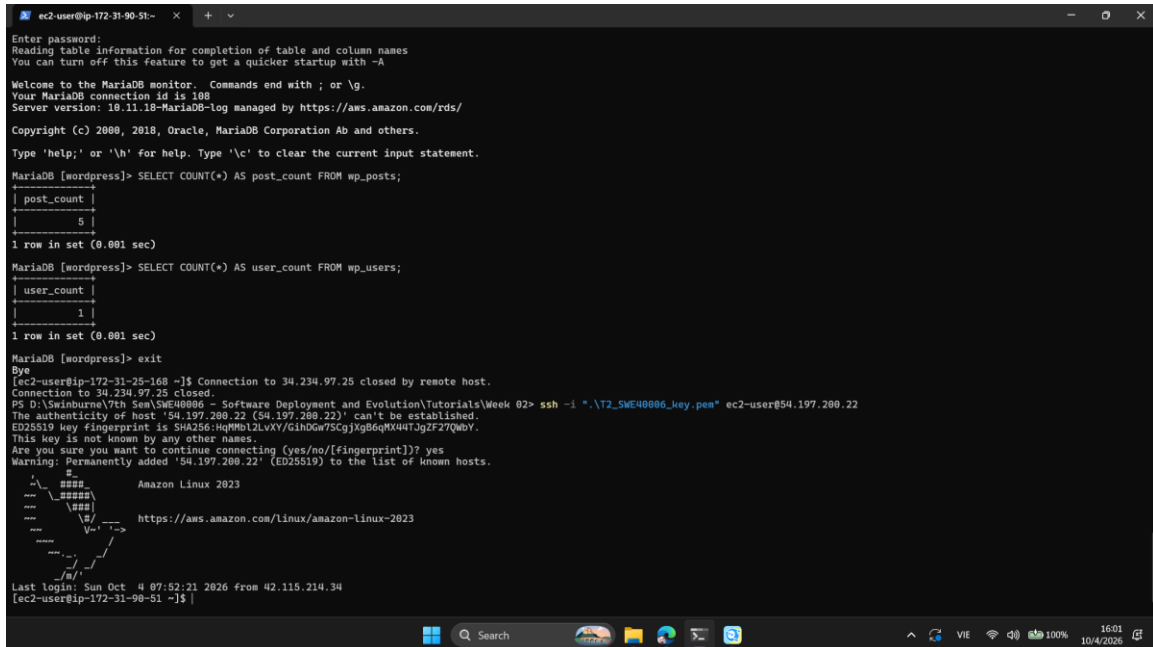


Fig. 12. Successful SSH access to the restored EC2 instance.

The restored instance confirmed that the backup could be used to recreate the WordPress server environment.

5. Application Load Balancer

An internet-facing Application Load Balancer named T2-swe40006-ALB was created with an HTTP listener on port 80. The target group T2-SWE40006-TG was configured to forward requests to the WordPress EC2 instance. After the listener was attached, the target became healthy and WordPress was accessible through the ALB DNS name.

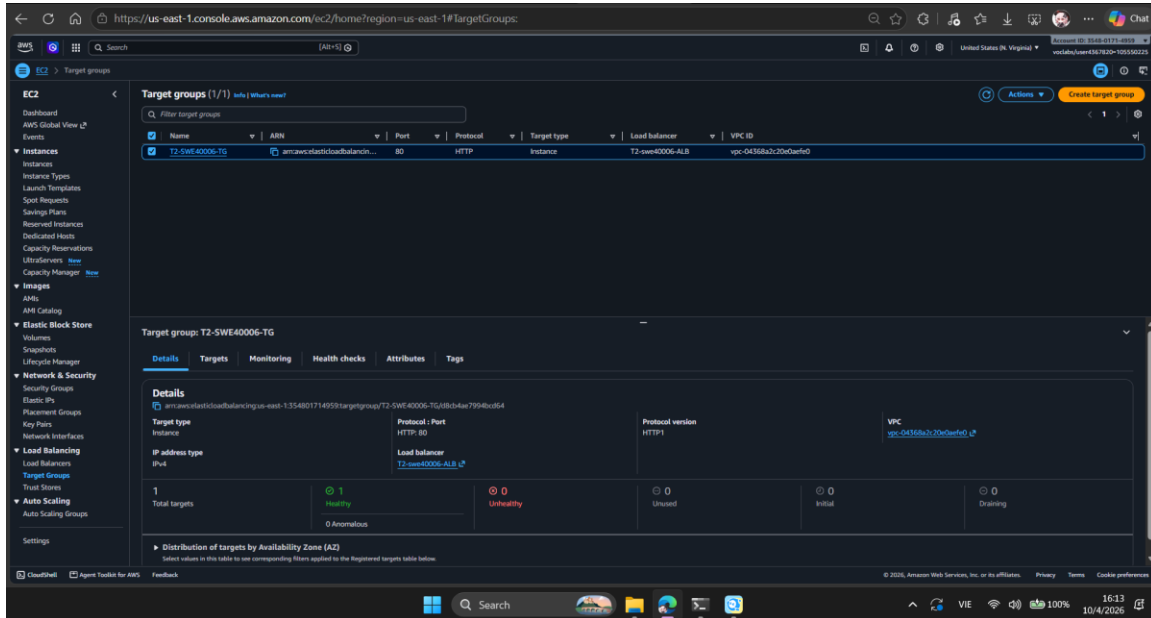


Fig. 14. Target group showing the WordPress EC2 target as healthy.

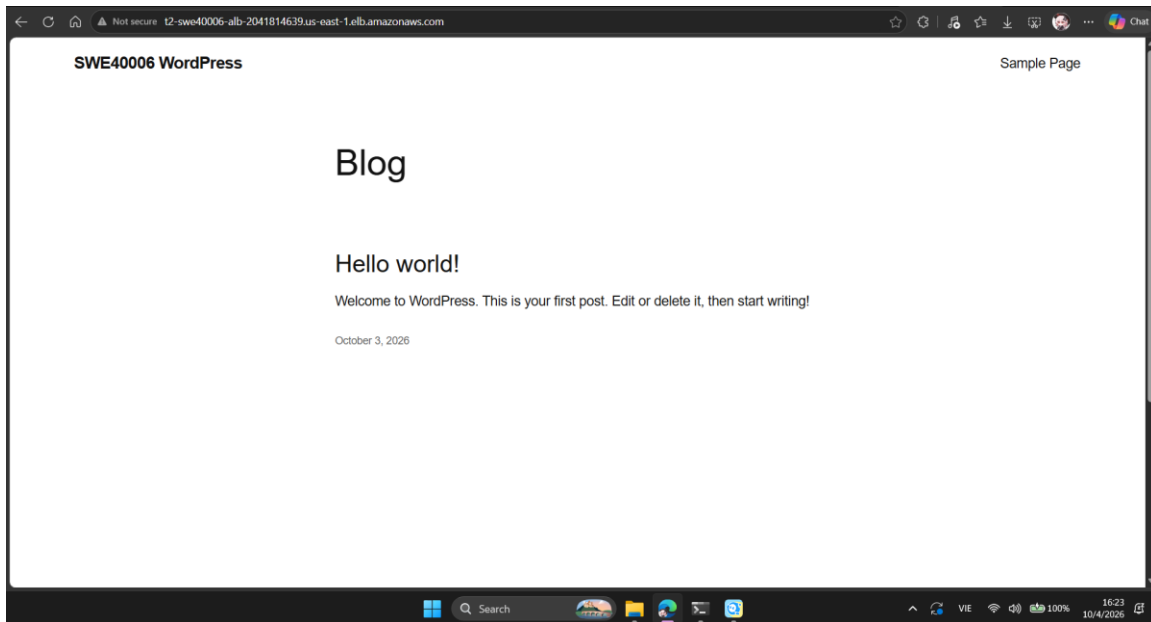


Fig. 15. WordPress successfully accessed through the ALB DNS endpoint using HTTP.

6. Launch Template and Auto Scaling

A Launch Template named T2_SWE40006_LT was created from the WordPress backup AMI. It specifies the t3.micro instance type, the T2_SWE40006_key key pair and the WordPress EC2 security group. The subnet was not fixed in the template so that the Auto Scaling Group could launch instances across the selected subnets.

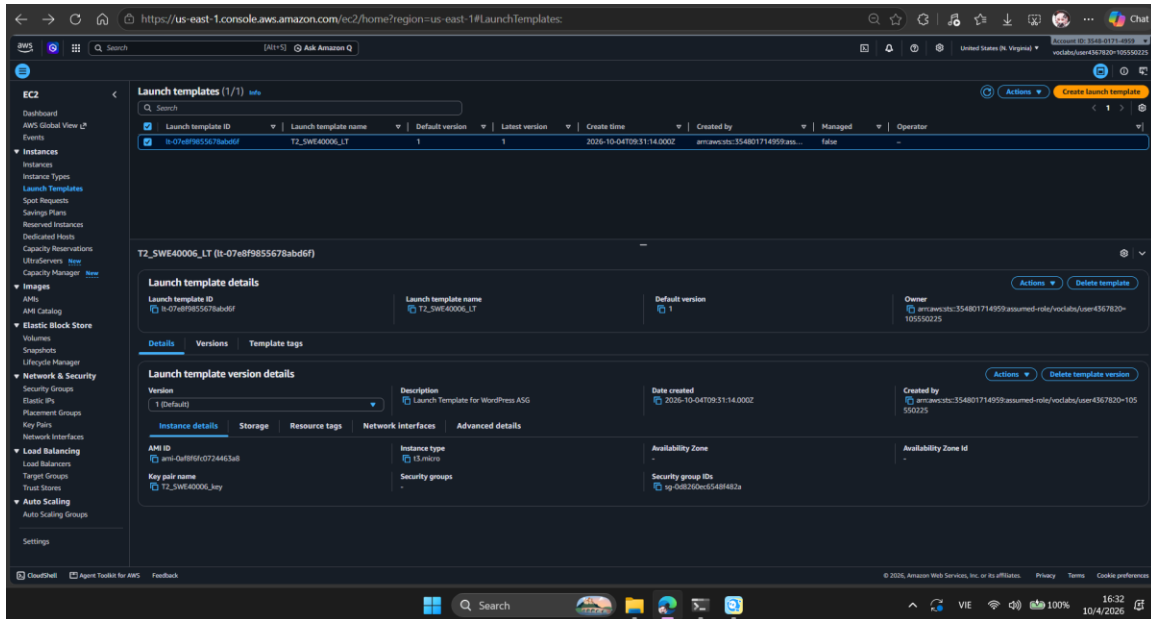


Fig. 16. Launch Template T2_SWE40006_LT created from the WordPress backup AMI.

The Auto Scaling Group T2_SWE40006_ASG was then created using the Launch Template and attached to the existing ALB target group. The group was configured with minimum capacity 1, desired capacity 1 and maximum capacity 2. CPU target tracking was configured at 50%.

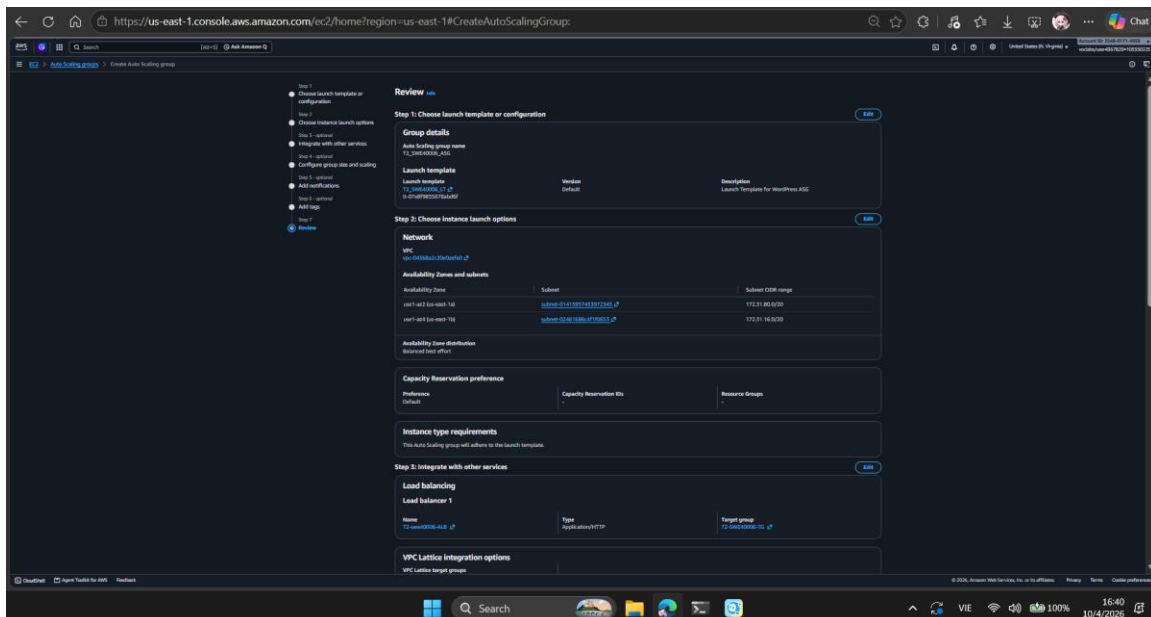


Fig. 17. Auto Scaling Group configuration showing the Launch Template, networking and load balancer attachment.

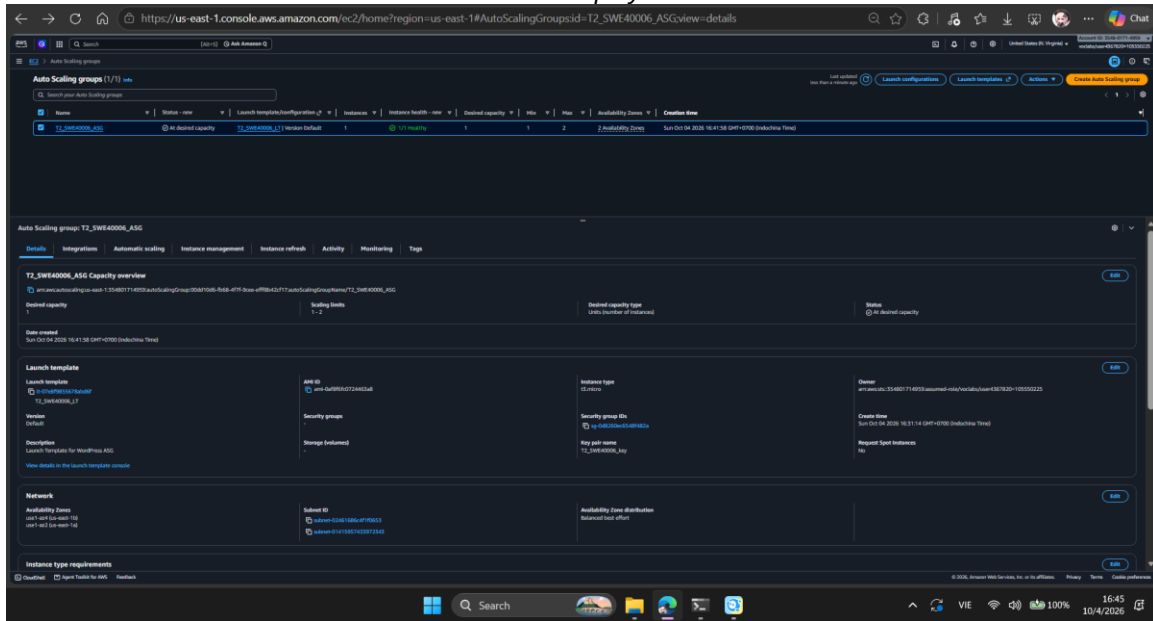


Fig. 18. Auto Scaling Group launched one healthy EC2 instance at the desired capacity.

To test scaling, the desired capacity was increased from one to two instances. The ASG launched the second instance and both instances were reported healthy. The target tracking policy then reduced desired capacity from two to one when CPU utilisation remained below the 50% target. This demonstrated the ASG scaling behaviour.

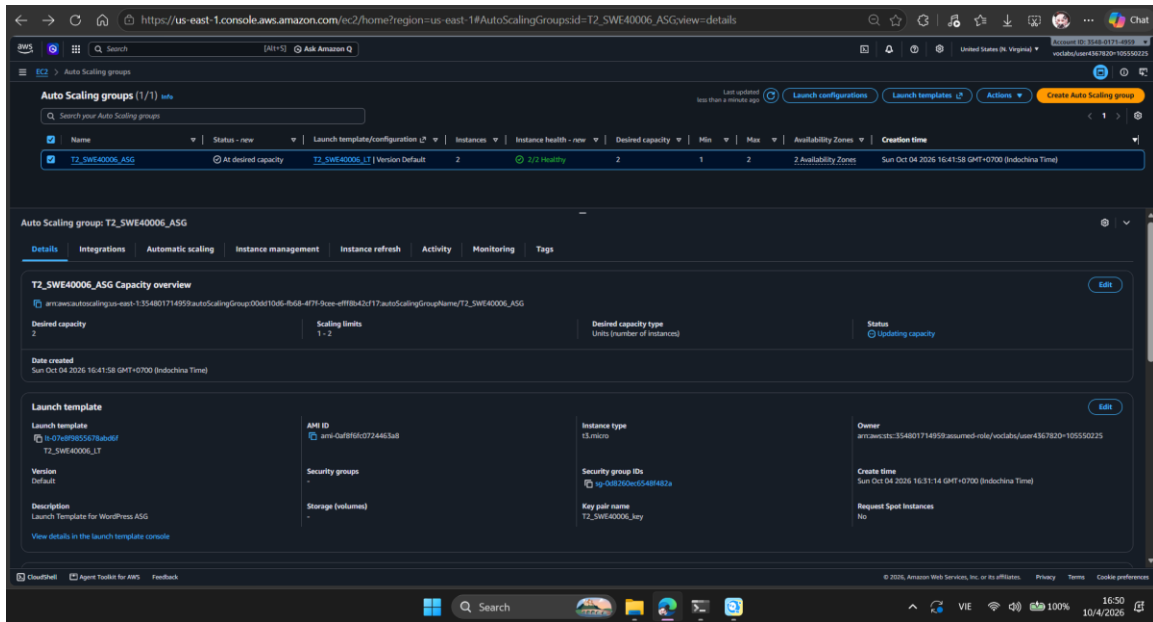


Fig. 19. Auto Scaling Group successfully running two healthy instances during the scale-out test.

SWE40006 - Software Deployment and Evolution

The screenshot shows the AWS Management Console for the Auto Scaling group T2_SWE40006_ASG. The console displays the following information:

- Auto Scaling groups (1/1):** A table showing the group's status as 'At desired capacity' with 2 instances. The group is healthy and has 2 availability zones.
- Auto Scaling group: T2_SWE40006_ASG:** A section for managing the group, including a search bar for notifications and a 'Send to' dropdown.
- Activity history (3):** A table showing the group's activity history. The table has columns for Status, Description, Cause, Start time, and End time. The activity history shows a sequence of events:
 - Connection draining in progress:** A status of 'In progress' with a description of 'Terminating EC2 instance: i-0a7b59737ab8f0a...' and a cause of 'At 2026-10-04T09:55:08Z a monitor alarm TargetTracking-T2_SWE40006_ASG-Alarm-aw-943D9618-B677-4c26-bb4c-2c201096 in state ALARM triggered policy Target Tracking Policy changing the desired capacity from 2 to 1. At 2026-10-04T09:55:12Z an instance was taken out of service in response to a difference between desired and actual capacity, shrinking the capacity from 2 to 1. At 2026-10-04T09:55:12Z instance i-0a7b59737ab8f0a was selected for termination.'
 - Successful:** A status of 'Successful' with a description of 'Launching a new EC2 instance: i-0b3b10d6754f3a38' and a cause of 'At 2026-10-04T09:49:11Z a user request update of AutoScalingGroup constraints to min: 1, max: 2, desired: 2 changing the desired capacity from 1 to 2. At 2026-10-04T09:49:25Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 1 to 2.'
 - Successful:** A status of 'Successful' with a description of 'Launching a new EC2 instance: i-0a7b59737ab8f0a' and a cause of 'At 2026-10-04T09:41:58Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 1. At 2026-10-04T09:42:03Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 1.'

Fig. 20. ASG activity history showing the scaling actions and target tracking policy.

The screenshot shows the AWS Management Console for the Auto Scaling group T2_SWE40006_ASG. The console displays the following information:

- Auto Scaling groups (1/1):** A table showing the group's status as 'At desired capacity' with 1 instance. The group is healthy and has 2 availability zones.
- Auto Scaling group: T2_SWE40006_ASG:** A section for managing the group, including a search bar for notifications and a 'Send to' dropdown.
- Activity history (3):** A table showing the group's activity history. The table has columns for Status, Description, Cause, Start time, and End time. The activity history shows a sequence of events:
 - Connection draining in progress:** A status of 'In progress' with a description of 'Terminating EC2 instance: i-0a7b59737ab8f0a...' and a cause of 'At 2026-10-04T09:55:08Z a monitor alarm TargetTracking-T2_SWE40006_ASG-Alarm-aw-943D9618-B677-4c26-bb4c-2c201096 in state ALARM triggered policy Target Tracking Policy changing the desired capacity from 2 to 1. At 2026-10-04T09:55:12Z an instance was taken out of service in response to a difference between desired and actual capacity, shrinking the capacity from 2 to 1. At 2026-10-04T09:55:12Z instance i-0a7b59737ab8f0a was selected for termination.'
 - Successful:** A status of 'Successful' with a description of 'Launching a new EC2 instance: i-0b3b10d6754f3a38' and a cause of 'At 2026-10-04T09:49:11Z a user request update of AutoScalingGroup constraints to min: 1, max: 2, desired: 2 changing the desired capacity from 1 to 2. At 2026-10-04T09:49:25Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 1 to 2.'
 - Successful:** A status of 'Successful' with a description of 'Launching a new EC2 instance: i-0a7b59737ab8f0a' and a cause of 'At 2026-10-04T09:41:58Z a user request created an AutoScalingGroup changing the desired capacity from 0 to 1. At 2026-10-04T09:42:03Z an instance was started in response to a difference between desired and actual capacity, increasing the capacity from 0 to 1.'

Fig. 21. ASG returned to one healthy instance after automatic scale-in.

7. Windows SSH Access and CLI Verification

For the HD requirement, SSH access was demonstrated from Windows PowerShell to the restored Linux EC2 instance using the T2_SWE40006_key.pem key. The connection succeeded and the Amazon Linux shell was used to run several verification commands.

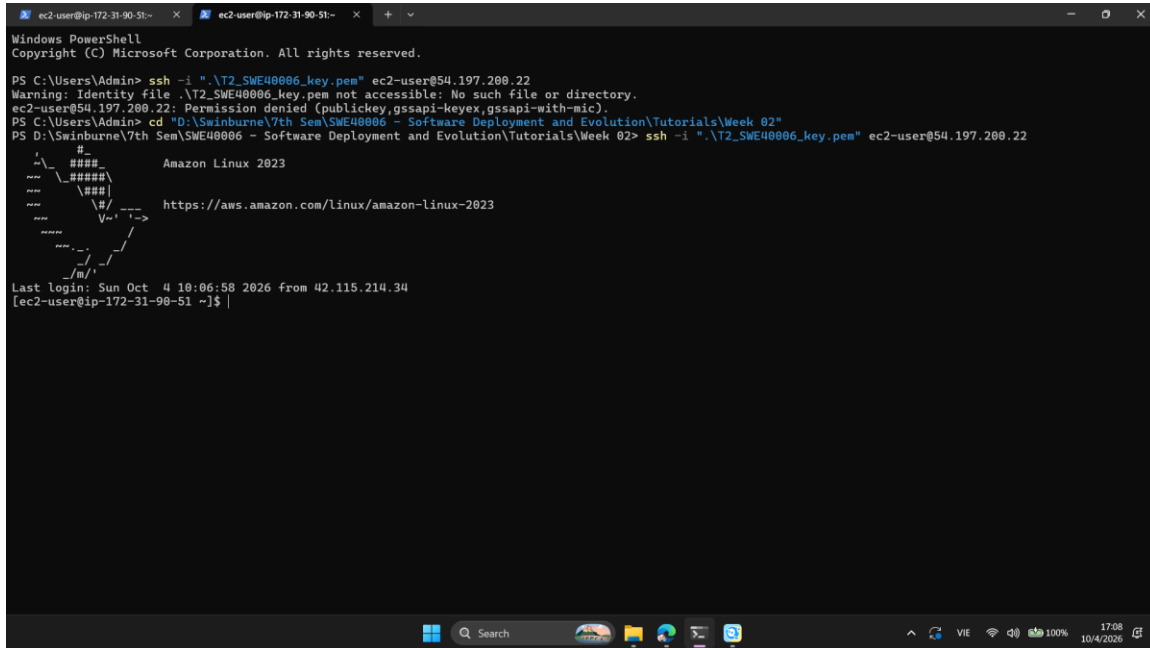


Fig. 22. Successful SSH connection from Windows PowerShell to the restored AWS EC2 instance.

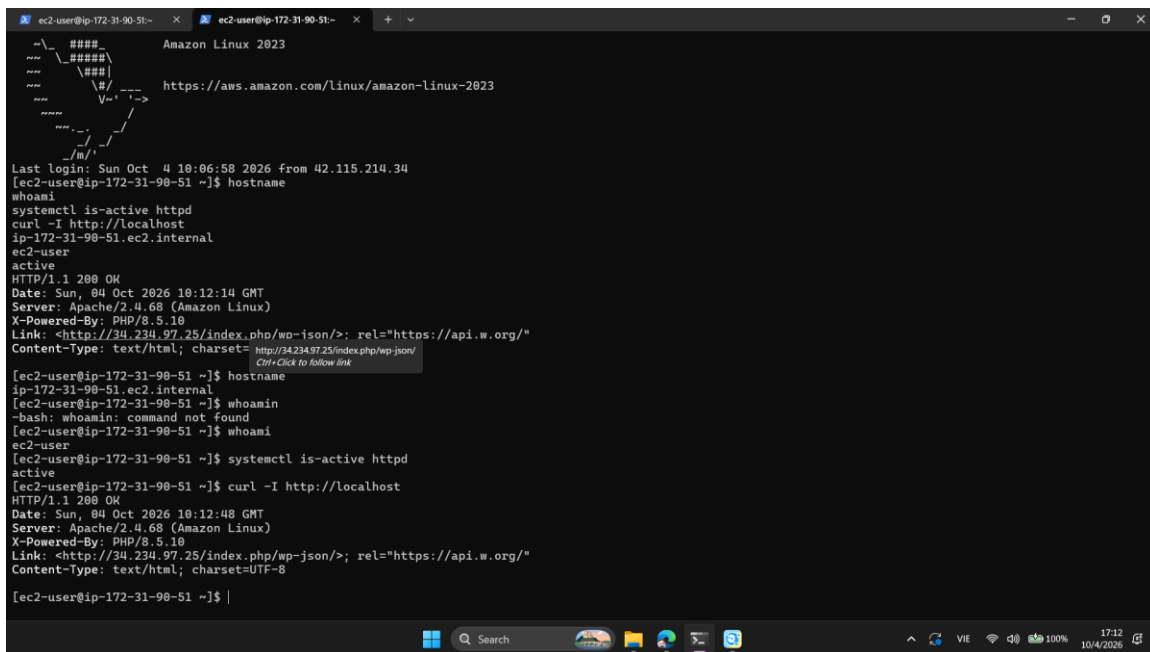


Fig. 23. Windows SSH CLI interactions verifying hostname, user, Apache service status and HTTP response.

The commands confirmed the instance hostname as ip-172-31-90-51.ec2.internal, the logged-in user as ec2-user, the Apache service status as active and the local HTTP response as HTTP/1.1 200 OK. A minor command-entry error, whoamin, was also identified and corrected to whoami.

8. Errors and Resolutions

The deployment issues encountered during Task 2 were investigated and corrected rather than omitted from the report.

a) Error 01 - SSH key path not found

Problem: The first SSH command was executed from a directory that did not contain T2_SWE40006_key.pem. PowerShell reported that the identity file could not be found and the SSH authentication failed.

Resolution: The working directory was changed to the folder containing the key file. The same SSH command was then executed successfully.

```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\WINDOWS\System32> dir "D:\Swinburne\7th Sem\SWE40006 - Software Deployment and Evolution\Tutorials\Week 02\T2_SWE40006_key.pem"

Directory: D:\Swinburne\7th Sem\SWE40006 - Software Deployment and Evolution\Tutorials\Week 02

Mode                LastWriteTime         Length Name
----                -
-a-----         10/3/2026   09:24             1674 T2_SWE40006_key.pem

PS C:\WINDOWS\System32> ssh -i "D:\Swinburne\7th Sem\SWE40006 - Software Deployment and Evolution\Tutorials\Week 02\T2_SWE40006_key.pem" ec2-user@18.212.78.182
Bad permissions. Try removing permissions for user: NT AUTHORITY\Authenticated Users (S-1-5-11) on file D:/Swinburne/7th Sem/SWE40006 - Software Deployment and Evolution/Tutorials/Week 02/T2_SWE40006_key.pem.
Warning: UNPROTECTED PRIVATE KEY FILE!
Permissions for 'D:\\Swinburne\\7th Sem\\SWE40006 - Software Deployment and Evolution\\Tutorials\\Week 02\\T2_SWE40006_key.pem' are too open.
It is required that your private key files are NOT accessible by others.
This private key will be ignored.
Load key "D:\\Swinburne\\7th Sem\\SWE40006 - Software Deployment and Evolution\\Tutorials\\Week 02\\T2_SWE40006_key.pem": bad permissions
ec2-user@18.212.78.182: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
PS C:\WINDOWS\System32>

```

Fig. 24. Initial SSH failure caused by an incorrect local key-file path.

b) Error 02 - WordPress database connection failure

Problem: During the database migration, WordPress temporarily displayed an error establishing a database connection while the external RDS configuration was being completed.

Resolution: RDS connectivity was tested from EC2, the WordPress database was imported into RDS, the RDS WordPress user privileges were configured and wp-config.php was updated to use the RDS endpoint. WordPress was then retested successfully.

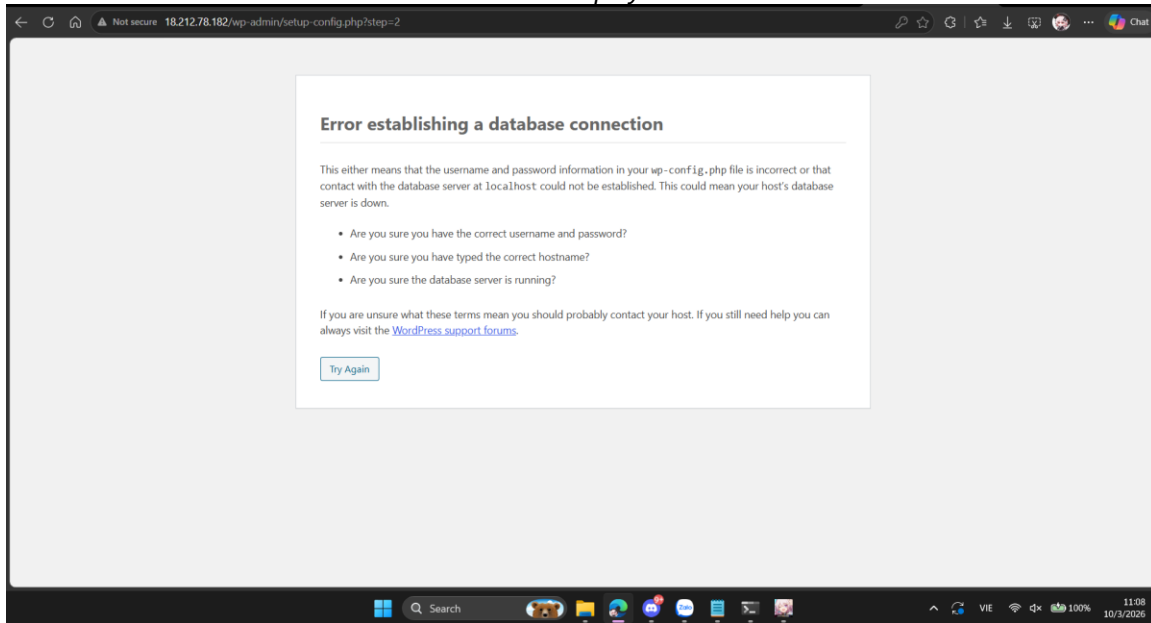


Fig. 25. WordPress database connection error encountered during the RDS migration.

c) Error 03 - ALB endpoint initially unreachable

Problem: The ALB endpoint initially appeared unreachable when it was opened with the wrong protocol. The browser displayed a connection timeout.

Resolution: The ALB listener was checked and confirmed to use HTTP on port 80. The endpoint was then accessed explicitly with http:// and WordPress loaded successfully through the load balancer.

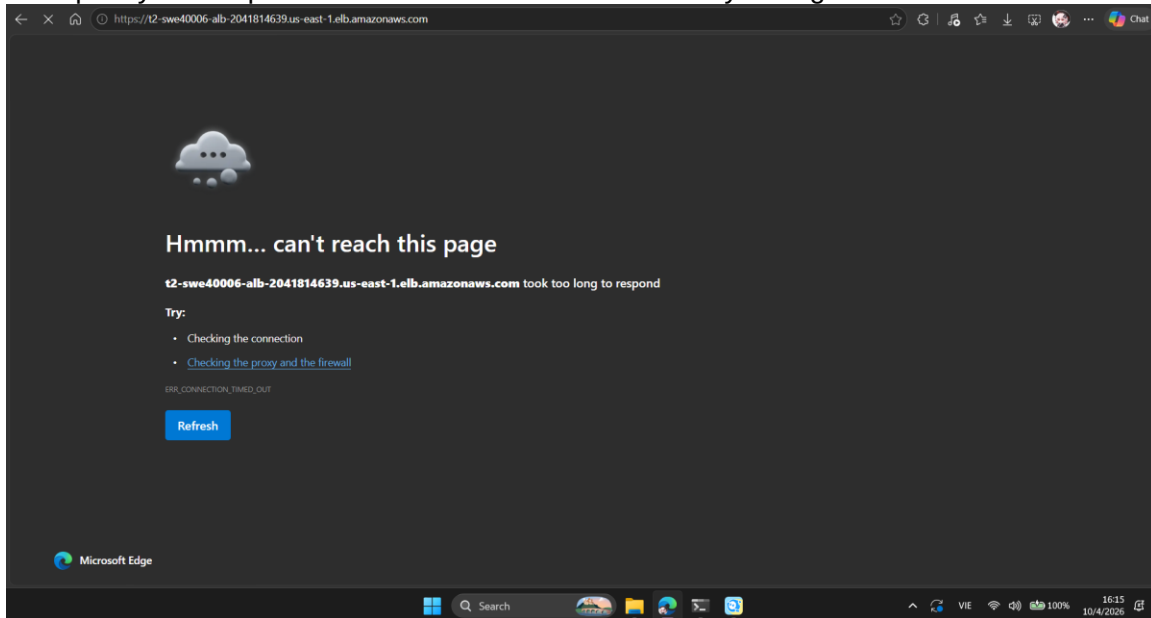


Fig. 26. Initial ALB access error before using the correct HTTP listener.

d) Error 04 - Incorrect CLI command

Problem: During the HD CLI verification, whoamin was entered instead of whoami and Bash returned command not found.

Resolution: The command was corrected to whoami and returned ec2-user. The remaining verification commands completed successfully.

9. Public Verification Endpoint and Deployment Resources

The deployed WordPress service can be verified publicly through the Application Load Balancer:

Public URL: <http://t2-swe40006-alb-2041814639.us-east-1.elb.amazonaws.com>

The restored EC2 instance used for SSH verification had public IPv4 address 54.197.200.22 at the time of testing. The ALB DNS endpoint is the preferred public verification endpoint because it represents the load-balanced deployment.

S3 backup bucket	Created in us-east-1
Resource	Configuration / identifier
Region	us-east-1
EC2 key pair	T2_SWE40006_key
EC2 security group	T2_SWE40006_SG
RDS endpoint	t2-swe40006-rds.ckxmbzgkctcaz.us-east-1.rds.amazonaws.com
Target group	T2-SWE40006-TG
Load balancer	T2-swe40006-ALB
Launch Template	T2_SWE40006_LT
Auto Scaling Group	T2_SWE40006_ASG
EC2 backup AMI	T2-SWE40006-WordPress-Backup

10. Source Code and Deployment Commands

Task 2 did not require custom application source code. The WordPress application and deployment infrastructure were configured using AWS services and Linux command-line tools. The command-line interactions used for verification are shown in the report screenshots. No binaries or private credentials are included in the report.

11. Conclusion

The WordPress deployment was successfully completed on AWS at Level 2.4. The deployment progressed from a single EC2 instance to an external RDS database, an Application Load Balancer, backup and restore, a Launch Template and an Auto Scaling Group. Scaling was tested from one to two instances and the target tracking policy subsequently performed automatic scale-in. Windows PowerShell SSH access and multiple CLI interactions were also demonstrated. The deployment issues encountered during the work were investigated, corrected and retested.